HW 6 due next Monday, Mar 23.

Quiz 8 coming soon.

Photo: https://www.bankofengland.co.uk/banknotes/polymer-50-pound-note

Class 16: Variants of Turing Machines

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Turing Machines

Two-tape

Random-Access Machines

Turing Completeness

Textbook [TCS] Section 7 and 8

https://introtcs.org/public/lec_06_loops.html
Loops and infinity

Recap: Turing Machine and Execution

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet – finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \to [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

Definition. The execution of a TM, $M = (\Sigma, C, \delta)$ on input $x \in \{0,1\}^*$ is this process:

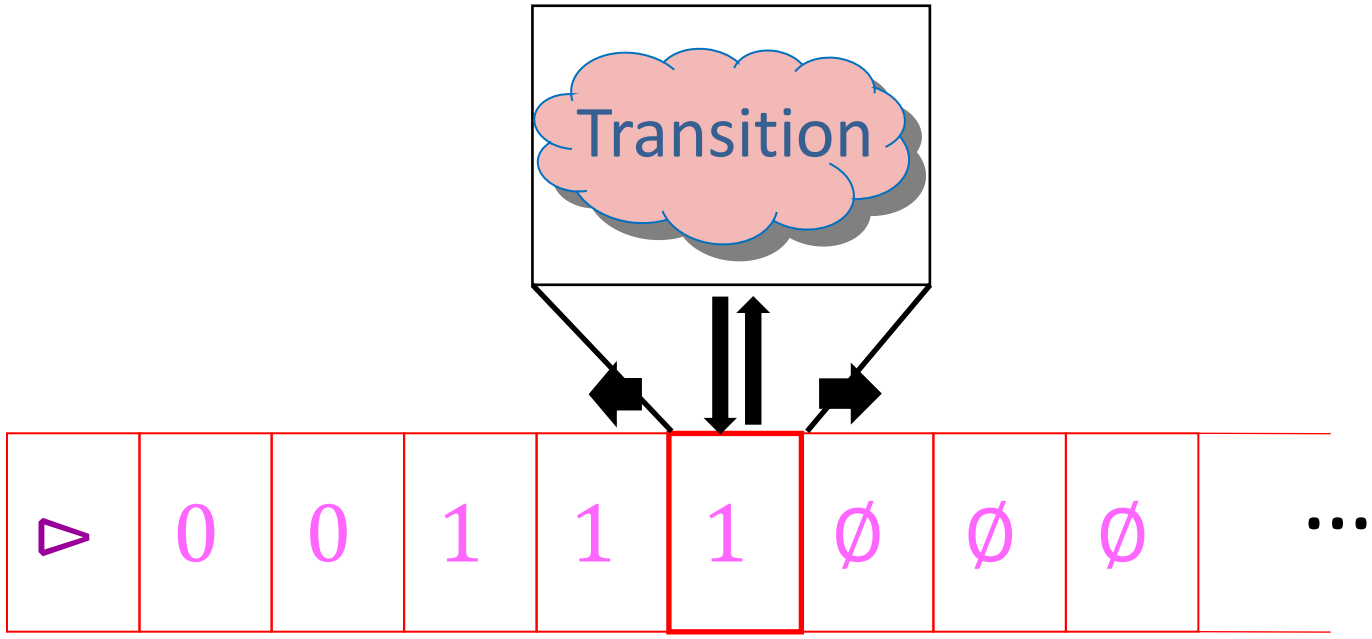
1. Initialize T as $\triangleright, x_0, x_1, \ldots, x_{n-1}, \emptyset, \emptyset, \emptyset, \ldots$ where $n = |x|$.

   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize two natural number variables, $i = 0, s = 0$.
3. **repeat**
   1. $(s', \sigma', D) = \boldsymbol{\delta}(s, T[i])$
   2. $s := s', T[i] := \sigma'$
   3. if D = $\mathbf{R}$: $i := i + 1$

      if D = $\mathbf{L}$: $i := \max\{i - 1, 0\}$

      if D = $\mathbf{H}$: **break**

Turing Machine



Tape initially contains the input followed by blanks

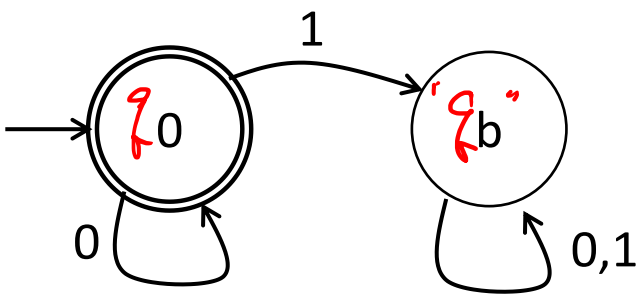
If transition "halts", → tape contains the output

Semi-infinite tape (memory)

Variants of TM Models

Example: From DFA to Turing Machines

- DFA



A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet – finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \to [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

writing output

$C = 2$? accept , reject

$\Sigma = \{ 0, 1, \triangleright, \emptyset \}$

Current state	Current symbol	Next state	Write symbol	Move direction
q_0	0	q_0	0	R
q_0	1	q_b	1	R
	$\emptyset$	(q_0)	($\emptyset$)	H
q_b	0	q_b	0	R
	1	q_b	1	R
	$\emptyset$	(q_b)	($\emptyset$)	H

Why is the *alphabet* finite?

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

will be agreed that the two-dimensional character of paper is no essential of computation. I assume then that the computation is carried out on one-dimensional paper, *i.e.* on a tape divided into squares. I shall also suppose that the number of symbols which may be printed is finite. If we were to allow an infinity of symbols, then there would be symbols differing to an arbitrarily small extent†. The effect of this restriction of the number of symbols is not very serious. It is always possible to use sequences of symbols in the place of single symbols. Thus an Arabic numeral such as 17 or 999999999999999 is normally treated as a single symbol. Similarly in any European language words are treated as single symbols (Chinese, however, attempts to have an enumerable infinity of symbols). The differences from our point of view between the single and compound symbols is that the compound symbols, if they are too lengthy, cannot be observed at one glance. This is in accordance with experience. We cannot tell at a glance whether 9999999999999999 and 999999999999999 are the same.

Why is the *number of states* finite?

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

Number of States of Mind

Follow simple rules
Remember what you are doing

"For the present I shall only say that the justification lies in the fact that the ***human memory is necessarily limited***."

Why is the *number of states* finite?

The behaviour of the computer at any moment is determined by the symbols which he is observing, and his "state of mind" at that moment. We may suppose that there is a bound B to the number of symbols or squares which the computer can observe at one moment. If he wishes to observe more, he must use successive observations. We will also suppose that the number of states of mind which need be taken into account is finite. The reasons for this are of the same character as those which restrict the number of symbols. If we admitted an infinity of states of mind, some of them will be "arbitrarily close" and will be confused. Again, the restriction is not one which seriously affects computation, since the use of more complicated states of mind can be avoided by writing more symbols on the tape.

Alternative TN Model: Two-Way Infinite Tape

⊳	0	0	1	1	1	∅	∅	∅	…

Semi-infinite tape

…	∅	∅	∅	∅	∅	∅	∅	∅	⊳	0	0	1	1	1	∅	∅	∅	…

Two-way infinite tape

Execute TM with Two-Way Infinite Tape

Definition. The **output** of the execution of a TM, $M = (\Sigma, k, \delta)$ is the result of this process:

1. Initialize $T[i] = \emptyset$ for all $i \in \mathbb{Z}$.
2. Initialize two natural number variables, $i = 0, s = 0$.
3. **repeat**
    1. $(s', \sigma', D) = \boldsymbol{\delta}(s, T[i])$
    2. $s := s', T[i] := \sigma'$
    3. if D = $\mathbf{R}$: $i := i + 1$
       if D = $\mathbf{L}$: $i := i - 1$
       if D = $\mathbf{H}$: **break**
4. If the process finishes (the repeat breaks), the result of this process is:
   $M(x) = T[1], \ldots, T[m]$ where $m > 0$ is the smallest integer, $T[m+1] \notin \{0,1\}$.
   Otherwise (i.e., the machine does NOT halt), then define $M(x) = \perp$.

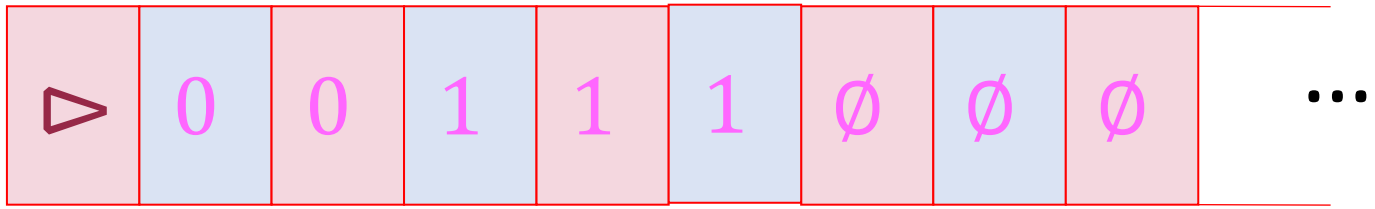
Reasons for (or not for) Two-Way Tape

Output?

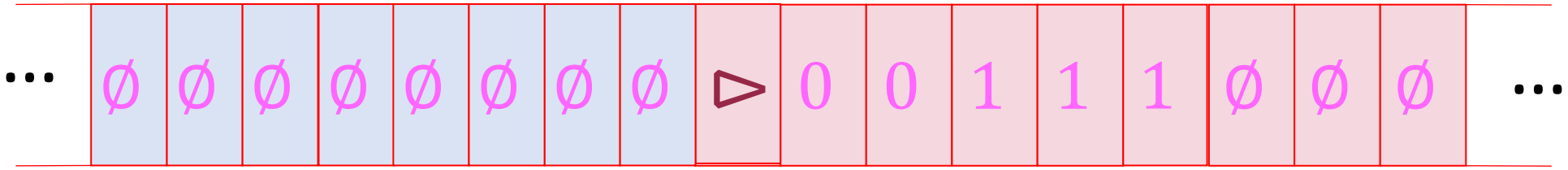
Write programs?

Does two-way differ from the one-way TM model?

Positive / Negative are mapped to Even / Odd



Semi-infinite tape

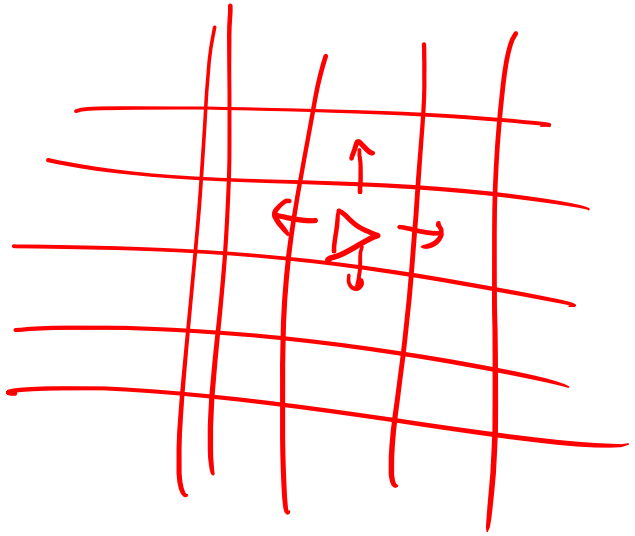


Two-way infinite tape

Omitted: To modify the transition function (move 2 grids, u-turn at 0, ….)

More Turing Machine models

- 2, 3, 4, or a constant number of tapes, each moves L/R independently
- Large (constant) alphabet
- Output by state



- 2-dimensional tape?!

- Move tape by arbitrary distance x?!

More Turing Machine models

- 2, 3, 4, or a constant number of tapes, each moves L/R independently
- Large (constant) alphabet
- Output by state

- 2-dimensional tape?!

All of them can be simulated by 1-tape TM

- Move tape by arbitrary distance x?!

This is Random-Access Machine, to discuss later

TM vs
Other Computation Models

Recap: DFA, NFA, Regular Expressions

All *equivalent* in computability:

For any language L, the following are the same:

- Exists DFA that accepts L
- Exists NFA that accepts L
- Exists regular expression that accepts L

Program P_1 equivalent to P_2

$\forall x, P_1(x) = P_2(x)$

Compare two Models

- Model A is *no stronger than* Model B:
  For any function F can be computed by model A, it holds that F can be computed by model B.
- Model A is *equivalent to* Model B:
  A is no stronger than B and B is no stronger than A.

Example:

DFA-Fun $\subseteq$ Reg-Fun

Reg-Fun $\subseteq$ DFA-Fun

DFA is equivalent to Reg

TM vs (idealized) Python

For any function F that can be computed by TM, can we compute F by Python?

(Real-world python run on finite machine)

Idealized Python: Infinite-bit int, infinite length array.

TM vs (idealized) Python

For any function F that can be computed by TM, can we compute F by Python?

Real-world python: No (run on finite machine).

Idealized Python: Infinite-bit int, infinite length array.

Yes! Write a python prog that *simulate* the TM and any TM.

TM $\subseteq$ Ideal-Python

HW 6

```
for step_i in range(max_step):
    assert self.transition[cur_state].keys() == self.alphabets
    assert head < len(tape)
    assert tape[head] in self.alphabets
    (next_state, write_symb, next_dir) = self.transition[cur_state][tape[head]]
    cur_state = next_state
    tape[head] = write_symb
    if next_dir == 'L':
        head -= 1
        if head < 0:
            head = 0
    elif next_dir == 'R':
        head += 1
        if head >= len(tape) - 1:
            tape.append('∅')
    elif next_dir == 'H':
        break
```

TM is no stronger than Most Programming Languages

Which prog. lang. can simulate TM?

(We just need inf loop and inf array)

TM

Is Ideal-Python no stronger than TM?

Can we show that TM $\supseteq$ Ideal-Python?

easy:
TM $\subseteq$ Python

Want: $\forall$ Python prog P
write a TM M s.t.

$P(x) = M(x)$ all x

Two-Tape TM and Random-Access Machines

Two-Tape Turing Machine

The transition reads, writes, and moves two tapes arbitrarily.

$\delta(st, \sigma)$:

next state,

σ_1 ~~to~~ to write

δ (st , read sym 1, read sym 2):

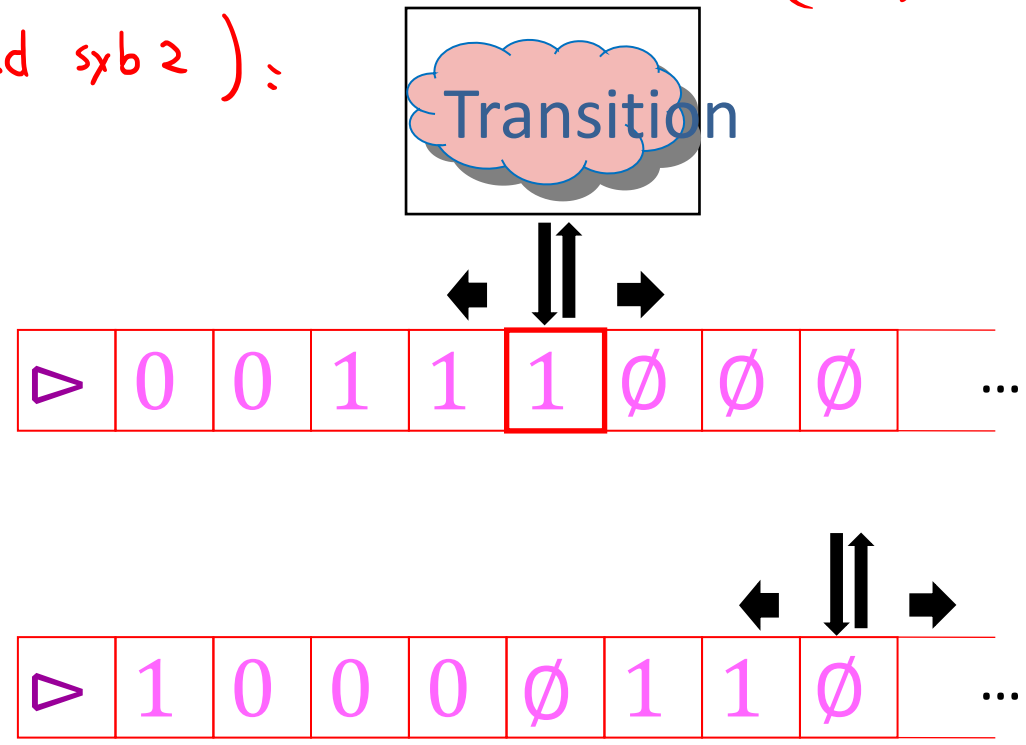
st next

write sym 1

write sym 2

move dir 1

move dir 2

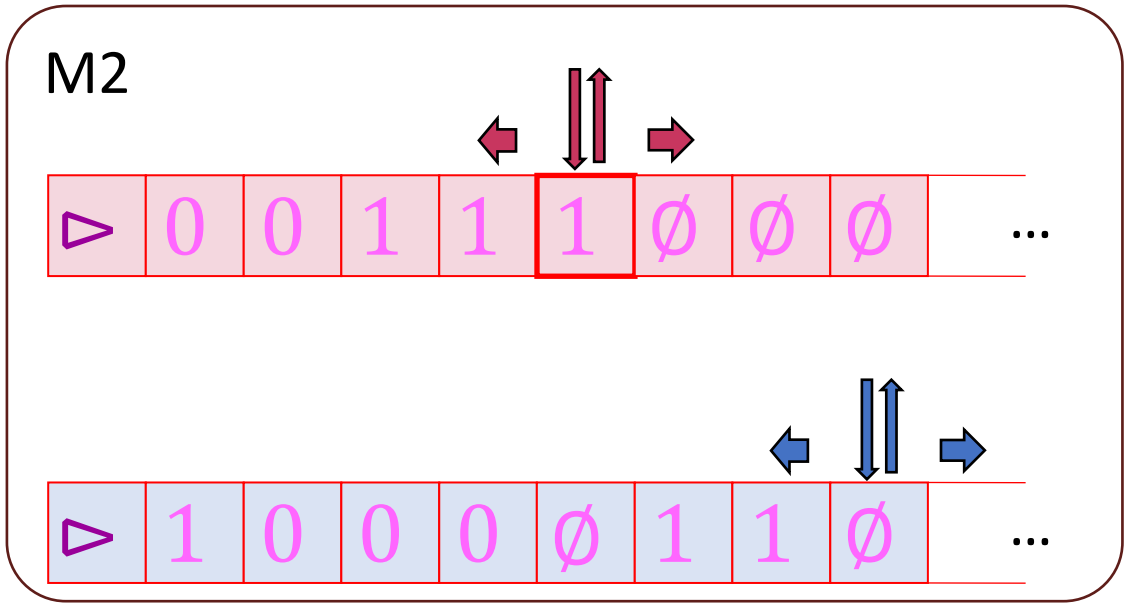


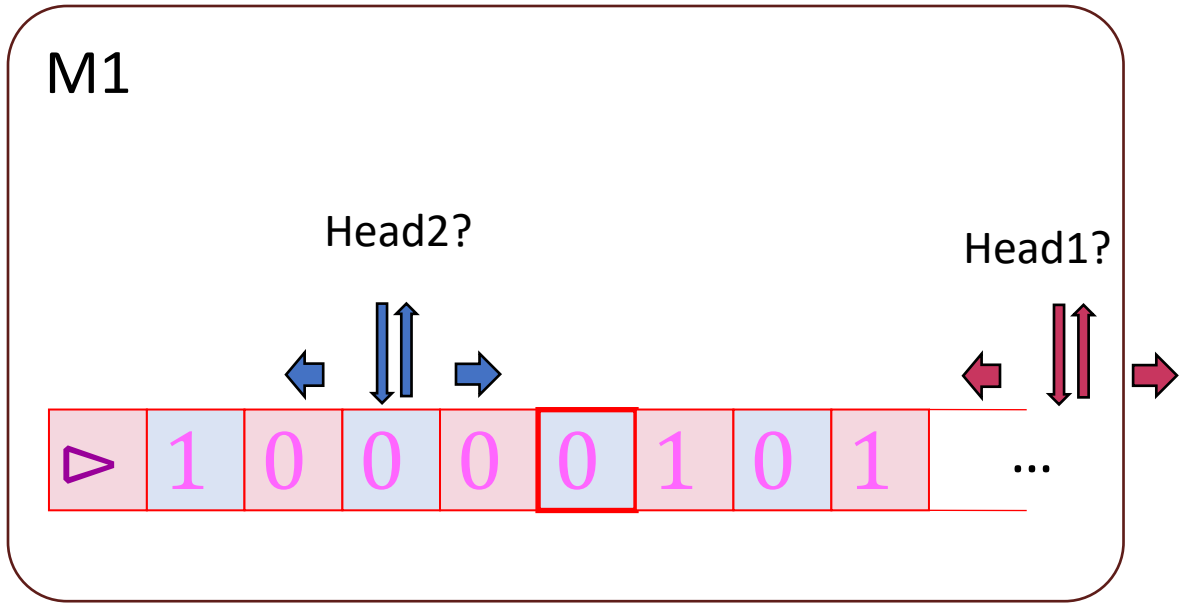
One-Tape TM is *Equivalent to* Two-Tape TM

Easy: One-tape-TM ⊆ Two-tape-TM

More work: One-tape-TM ⊇ Two-tape-TM

Proof: For each Two-tape-TM M2, build a One-Tape-TM M1 s.t. M1(x) = M2(x) for all input x.





One-Tape TM is *Equivalent to* Two-Tape TM

Easy: One-tape-TM ⊆ Two-tape-TM

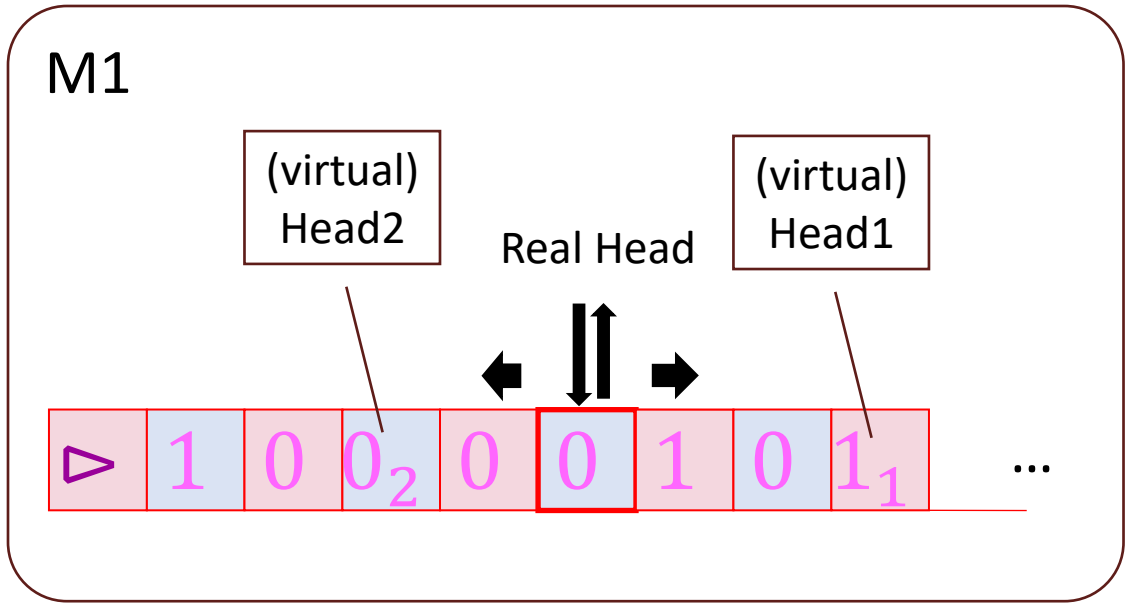
More work: One-tape-TM ⊇ Two-tape-TM

Proof: For each Two-tape-TM M2, build a One-Tape-TM M1 s.t. M1(x) = M2(x) for all input x.

Ideas:

Interleave two tapes into one.

Use special symbol for “head”.



Random-Access Machine (RAM)

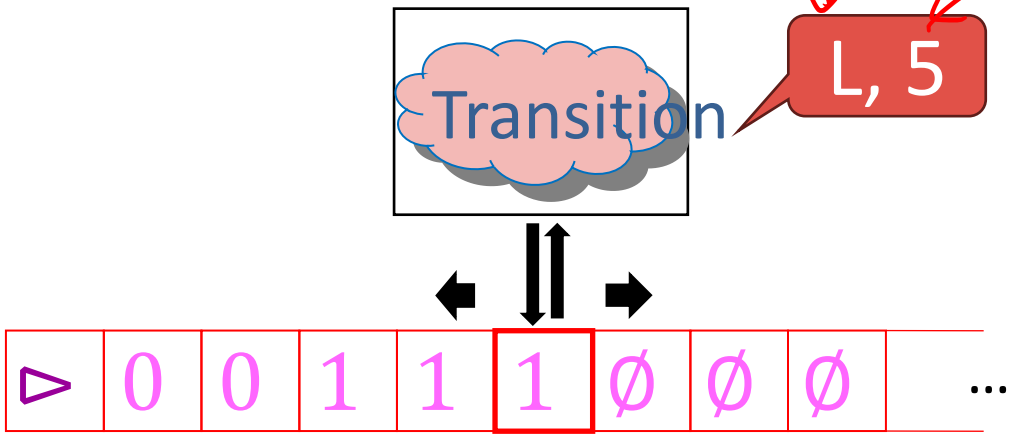
One transition can move tape left/right for k grids,

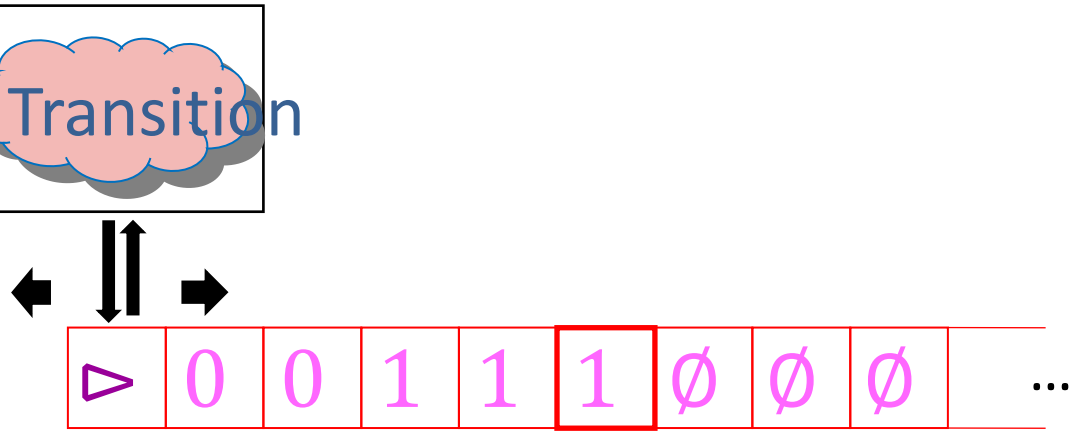
in one step.

which dir

how far

How to define it?





Defining Random-Access Machine (RAM)?

A *RAM* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\} \times \mathbb{N}$$

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

Defining Random-Access Machine (RAM)?

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet – finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

A *RAM* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet – finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

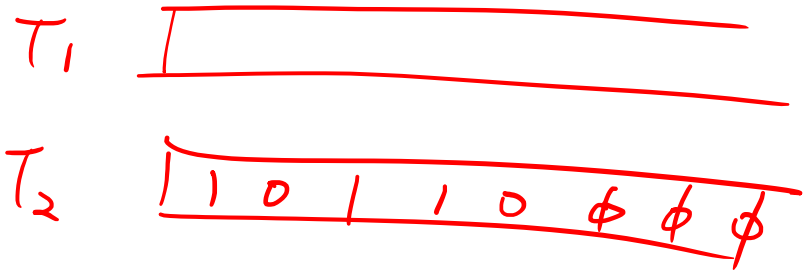
$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\} \times \mathbb{N}$$

Only difference.
What's wrong?

WK's Def: Random-Access Machine (RAM)

A *RAM* is defined by a Two-Tape Turing Machine, but it is augmented with **J** (jump) in addition to L/R/S/H.
During execution on J, Tape2 is interpreted as a (potentially negative) integer k, and Tape1 is moved by k grids.

Another def: jump no more than B distance for some finite B.



Detailed definition unimportant

Section 8.1 gives another informal definition:
https://introtcs.org/public/lec_07_other_models.html#ram-machines-and-nand-ram

RAM is equivalent to One-Tape TM

Easy: RAM $\supseteq$ Two-Tape-TM

Doable: RAM $\subseteq$ Two-Tape-TM
(by writing a two-tape TM)

Detailed proof unimportant

RAM $\equiv$ Two-Tape-TM

RAM $\equiv$ One-Tape-TM

Logistics

- Explain your solutions ☺
- Midterm2 next Tuesday, Mar 24

HW 6 due next Monday, Mar 23.

Quiz 8 coming soon

Circuits

Plan

Turing machine

Computability

Textbook [TCS] Section 7

https://introtcs.org/public/lec_06_loops.html
Loops and infinity